

Can a small model solve harder problems by dividing the work?

Preliminary research and questions for discussion

We can teach a useful routine.

Training improved how the model gathers information and calculates an answer.

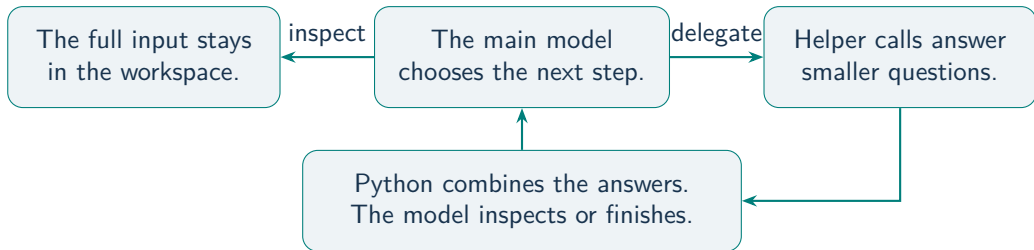
The connections also matter.

A good answer to a small question can be lost or misused when answers are combined.

Our next goal is to turn these observations into a narrow, reproducible research contribution.

Alex Towell | Advisor discussion | These are preliminary findings, not a general solution.

An RLM gives the model a workspace and a way to ask smaller questions.



The main model need not read every record itself. A helper can divide its own task again.

RLM means recursive language model. Our tests mostly use one layer of helper calls.
Concept: [Alex Zhang's blog](#) and [Recursive Language Models](#).

The helper reads the text; Python does the counting.

For each user, add the weights of their place questions. How many users have a total above 5?

User	Question	Weight	Place question?
Ada	Where is Oslo?	4	Yes
Ada	What is the capital of Peru?	3	Yes
Ben	Who wrote Hamlet?	6	No
Ben	Where is Kyoto?	2	Yes

Python computes Ada: $4 + 3 = 7$ and Ben: 2.

Only Ada exceeds 5, so the answer is **1 user**.

Simplified illustration, not a quoted training example. Text-reading errors and calculation errors are different.

We trained the model on examples of actions, not just final answers.

What the model observes

The question, a path to the records, and basic information about the input.

What it observes next

The helper's actual labels and the result returned by Python.

What the example teaches it to do

Read the records and ask the helper to classify them.

Choose the calculation that this question requires.

Run that calculation and return its answer.

Supervised fine-tuning (SFT) teaches demonstrated actions. Our data use public questions with added users and weights.

72 demonstrations; six passes through all 72. Helper mistakes were preserved. We trained a small set of added weights, not the full model. This is a starting skill, not open-ended planning.

Training gains appeared again with a different set of demonstrations.



Success means a correct answer from the requested calculation, not a lucky match.

Both runs restart from the same already-trained model; the third bar is not more training on the second. All use the same 72 questions, with changed names and numbers, on eight previously used inputs. Missing results: 17, 0, and 2. Even counting all missing baseline results as successes gives only 29. [S1, S2]

An answer can be right for the wrong record.

A simple reading task

Passage: “Maya bought a red bike.”

Row 7: “Maya bought a vehicle.”

The passage supports this statement.

Row 8: “The bike is blue.”

The passage contradicts this statement.

What if the answer slot for row 7 carries row 8's ID?

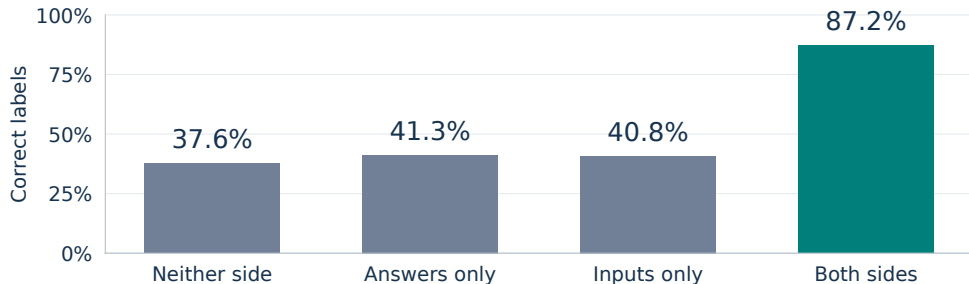
The model may answer for row 8 instead.

“Contradicted” is then a sensible label—attached to the wrong statement.

In our test, misleading record IDs reduced accuracy from about 80% to 39%.

The example is illustrative. In the actual 16-batch test, unrelated IDs gave 618/768 correct labels; IDs pointing to another visible record gave 298/768. Output IDs were forced by the format; the model chose the labels. This is evidence about behavior, not a proven internal mechanism. [Evidence H1]

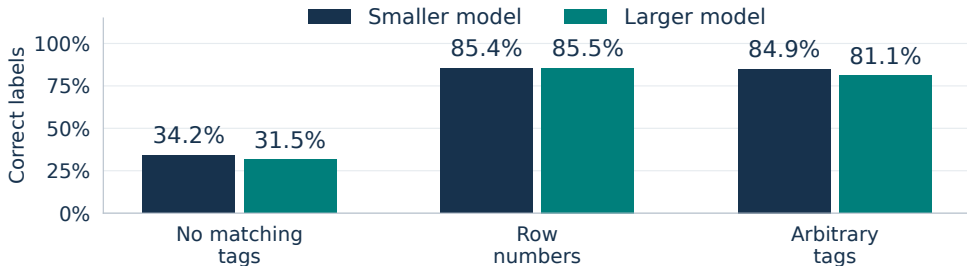
Matching row numbers on both sides helped the model keep answers aligned.



The benefit of matching both sides was positive in all 16 new batches.

We measured the later 32 of 48 statements, across three identifier conditions: 1,536 labels per bar. Batches, not repeated labels, are independent units. The format supplies the keys, not the answers. [H2]

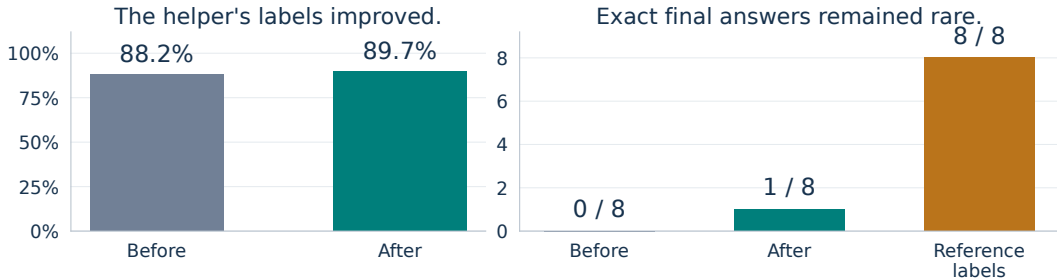
Matching tags helped both tested models, and the tags did not have to be numbers.



Ordinary numbers led for the larger model, but arbitrary tags also helped.

Two Qwen models, same 16 batches; 1,536 later labels per bar. Both gains appeared in every batch for both models. The software supplied tags and order, not labels. This does not isolate size or test a different model family. [H3, H4]

Better individual labels did not reliably produce the right combined answer.



Here we trained the helper. The plan and Python calculation were supplied.

1,280 label slots; eight problems sharing four inputs. Reference labels are a diagnostic control, not information given to the model. Overall numerical error did not improve. [C1]

This reward-training recipe did not improve the trained model.

We also let the model try solutions and rewarded correct final answers.

Before reward training

55 / 72

Verified correct final answers.

After six weight updates

47 / 72

Verified correct final answers.

This changes our next experiment, not our view of whether reward training can ever work.

This is a different readout and metric from slide 5: final-answer correctness alone. Missing results cannot reverse the decline (before: 55–56 possible; after: 47–50). One small model, one recipe, and a previously used panel. We will test a smaller update size and better ways to assess intermediate work. [Evidence R1]

The next experiments should test whether these gains survive a harder setting.

For the training result

Test the two trained models on new inputs and unfamiliar ways of combining the smaller tasks.

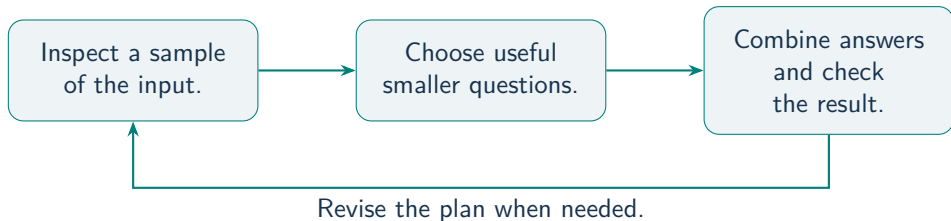
For the input–answer matching result

Test whether matching tags also improve the final combined answer. Then try a different task or model family.

A strong paper should explain when the improvement works and where it stops.

These are next tests, not findings. Numbered batch prompts already exist; our contribution must go beyond “numbering helps.”

The longer-term goal is to learn how to investigate and divide a new problem.



We can start with examples, then learn from checked outcomes. We can also improve the tools and instructions alongside the model.

This is a research direction, not an achieved capability. The related `rlm-bootstrap` project supplies ideas for this learning loop. Future comparisons must separate improvements in the model from improvements in its tools.

Which result should we turn into the next focused study?

What would make the finding convincing?

Should we prioritize a second model, a different task, or a clearer explanation of the failure?

Where would this matter most?

Is there a realistic task in your work where many small judgments must be combined reliably?

Our current recommendation is to connect the strong input–answer matching result to a useful final-task improvement.

The guide explains each slide. Companion documents give methods, unsuccessful experiments, and possible publication paths.